

TARG Architecture Relationship Report

This document summarizes the principal relationships among the classes in the TARG system, indicating which classes create, own (composition/aggregation), or depend on other classes.

1. Benchmark Subsystem

BenchmarkLoader

Creates: Benchmark, BenchmarkQuery.

Depends on: JSON benchmark file and benchmark model validation.

Used by: RetrievalRunner, RetrievalVerificationRunner, run_retrieval_benchmark.py, measure_retrieval_accuracy.py.

Benchmark

Owns (composition): BenchmarkQuery (1..*).

BenchmarkQuery

Consumed by RetrievalInspector, RetrievalRunner and RetrievalVerificationRunner.

2. Corpus Construction Subsystem

RepresentativeCorpusBuilder

Creates CorpusDocument; depends on the PDF loader and corpus directory; used by build_authoritative_vector_store.py.

Chunker

Depends on CorpusDocument; creates Chunk; uses ChunkMetadata.

ChunkMetadata

Provides metadata for Chunk and VectorStoreBuilder.

VectorStoreBuilder

Depends on Chunk, embedding model and vector store backend; creates the vector store.

3. Integrated Corpus Builder

RepresentativeCorpusBuilder (rag/corpus_builder.py)

Owns CorpusChunkingConfig and CorpusDocumentSpec; creates CorpusBuildResult.

CorpusBuildResult

Owns CorpusSourceSummary (1..*) and CorpusChunkingConfig.

4. Retrieval Subsystem

BaseRetriever

Abstract interface implemented by VectorRetriever.

VectorRetriever

Depends on vector store; creates RetrievalResult.

RetrievalResult

Owns RetrievedDocument (0..*).

5. Retrieval Inspection

RetrievalInspector

Depends on VectorRetriever and BenchmarkQuery; creates RetrievalInspection and RetrievedChunk.

RetrievalInspection

Owns RetrievedChunk (1..*).

RetrievalRunner

Aggregates BenchmarkLoader and RetrievalInspector; creates RetrievalInspection collections.

6. Retrieval Verification

RetrievalVerificationRunner

Aggregates BaseRetriever; depends on Benchmark and BenchmarkQuery; creates RetrievalVerification and RetrievalVerificationResult.

RetrievalVerification

Owns RetrievalBenchmark and RetrievalResult.

RetrievalVerificationResult

Owns RetrievalVerification (1..*).

7. Draft Generation

DraftGenerator

Depends on language model and TokenizerProtocol; creates DraftOutput.

DraftOutput

Owns DraftStep (1..*).

8. Margin Uncertainty

MarginUncertaintyScorer

Depends on DraftOutput; creates MarginUncertaintyResult.

MarginUncertaintyResult

Owns TokenMarginScore (1..*).

9. Answer Generation

AnswerGenerator

Depends on language model, TokenizerProtocol and optionally RetrievalResult; creates AnswerOutput.

10. Answer Evaluation

AnswerEvaluator

Implements BaseAnswerEvaluator; creates EvaluationResult.

EvaluationResult

Owns MetricResult (1..*).

MetricResult

Depends on EvaluationMetric.

11. Stage 2.5 Experiment

Stage2_5ExperimentRunner

Aggregates DraftGenerator, MarginUncertaintyScorer, AnswerGenerator, Retriever and AnswerEvaluator; creates RetrievalComparison and ExperimentResult; depends on DevelopmentRecord.

RetrievalComparison

Owns RetrievalResult, two AnswerOutput objects and two EvaluationResult objects; creates DevelopmentExample.

ExperimentResult

Owns RetrievalComparison (1..*).

12. Threshold Calibration

ThresholdCalibrator

Depends on DevelopmentExample; creates ThresholdEvaluation and CalibrationResult.

CalibrationResult

Consumed by RetrievalGate.

13. Retrieval Gate

RetrievalGate

Aggregates CalibrationResult; creates RetrievalDecision.

RetrievalDecision

Depends on RetrievalDecisionReason.

14. TARG Pipeline

TARGPipeline

Aggregates DraftGenerator, MarginUncertaintyScorer, RetrievalGate, BaseRetriever and AnswerGenerator; creates PipelineResult.

PipelineResult

Owns RetrievalResult and contains the draft, uncertainty result, gate decision, retrieval output and final answer.

15. Utility Modules

ModuleLoader

Provides dynamic module loading; used by measure_retrieval_accuracy.py.

build_authoritative_vector_store.py

Uses RepresentativeCorpusBuilder, Chunker and VectorStoreBuilder.

run_retrieval_benchmark.py

Uses BenchmarkLoader, RetrievalInspector and RetrievalRunner.

measure_retrieval_accuracy.py

Uses ModuleLoader, BenchmarkLoader, VectorRetriever and RetrievalVerificationRunner.

Architectural Summary

Layered Architecture

1. Knowledge Preparation Layer.
2. Retrieval Layer.
3. Generation Layer.
4. Experimentation Layer.
5. Application Layer.

The architecture is predominantly layered and acyclic, with entry-point scripts orchestrating the lower-level components without embedding domain logic.